

ARQUITECTURA HEXAGONAL

Ingeniería de Software

Guía de Referencia Académica

Profesor: Ing. Germán González Rozo

Tabla de Contenido

Cómo usar esta guía: (1) Lee primero las secciones 1–3 para entender la idea central. (2) Luego revisa la sección 4 para saber cómo probar tu código. (3) Finalmente, usa la sección 5 como 'menú de evolución' para practicar cambios sin romper el dominio. En el Trabajo Extra-Clase (sección 7) encontrarás instrucciones y sugerencias; si algo no se ajusta a tu contexto justifica tu decisión.

Tabla de Contenido.....	2
1. Introducción: El Problema de la Deuda Técnica y la Necesidad de Desacoplamiento	4
2. Anatomía de la Arquitectura: Diseccionando Cada Capa.....	6
2.1 El Dominio: El Núcleo Soberano.....	6
2.2 Los Puertos: El Contrato entre el Núcleo y el Exterior	8
2.2.1 Puertos de Entrada (Input Ports).....	8
2.2.2 Puertos de Salida (Output Ports).....	9
2.3 La Capa de Aplicación: Los Casos de Uso	10
2.4 Los Adaptadores: La Conexión con el Mundo Real	11
2.4.1 Adaptadores de Entrada (Driving / Primary Adapters).....	11
2.4.2 Adaptadores de Salida (Driven / Secondary Adapters)	12
2.5 La Regla de Dependencia: El Principio más Importante	13
3. Estructura Canónica del Proyecto	15
4. Estrategia de Pruebas: La Pirámide de Testing	16
4.1 Pruebas Unitarias del Dominio	17
4.2 Pruebas de Integración con Adaptadores en Memoria	18
5. Evolución de la Aplicación Apoyada en la Arquitectura Hexagonal	19
5.1 Evolución 1: Agregar un Nuevo Canal de Entrada.....	19
5.2 Evolución 2: Migración de Base de Datos (PostgreSQL → MongoDB).....	20
5.3 Evolución 3: Transición a Arquitectura de Microservicios	21
5.4 Evolución 4: Integración con Sistemas de IoT	21
6. Diagramación Formal Requerida	23
6.1 Modelo C4 — Nivel 2: Diagrama de Contenedores	23
6.2 Diagrama de Puertos y Adaptadores (Vista Hexagonal).....	25
7. Trabajo Extra Clase	27
7.1 Descripción General	27
7.2 Fases del Trabajo.....	27
Fase 1: Análisis del Monolito y Mapa de Deuda Técnica	27
Fase 2: Implementación del Dominio y Puertos	28
Fase 3: Implementación de la Capa de Aplicación y Adaptadores de Salida	28
Fase 4: Adaptadores de Entrada + Evolución Demostrativa	29

Fase 5: Documentación, Pruebas Finales y Preparación de Presentación	29
7.3 Entregables	30
Entregable 1: Repositorio con Dos Ramas	30
Entregable 2: Informe Técnico Final	30
Entregable 3: Presentación a la Clase	31
Entregable 4: Video de Demostración (Opcional genera 5 puntos adicionales a la nota final del corte)	31
7.4 Rúbrica de Evaluación.....	32
8. Banco de Preguntas de Reflexión y Sustentación.....	33
8.1 Preguntas Conceptuales	33
8.2 Preguntas de Diseño y Código	33
8.3 Preguntas de Evolución y Escalabilidad	33
9. Errores Comunes y Guía de Diagnóstico	35
10. Recursos Bibliográficos y Herramientas.....	36
10.1 Bibliografía Obligatoria	36
10.2 Bibliografía Complementaria	36
10.3 Herramientas Recomendadas	36
11. Nota Final del Docente.....	38

1. Introducción: El Problema de la Deuda Técnica y la Necesidad de Desacoplamiento

1.1 El Escenario Clásico: El Monolito Acoplado

En la industria del software, un patrón recurrente —especialmente en startups y proyectos heredados (legacy systems)— es la arquitectura monolítica fuertemente acoplada. En este modelo, la lógica de negocio, el acceso a datos, el enrutamiento HTTP y la interfaz de usuario conviven entrelazados en los mismos módulos o clases. El siguiente fragmento conceptual ilustra el problema:

```
# ❌ ANTI-PATRÓN: Sistema de pagos monolítico acoplado
from fastapi import FastAPI
from sqlalchemy import create_engine

app = FastAPI()
engine = create_engine('postgresql://...')

@app.post('/pago')
def procesar_pago(monto: float, cliente_id: int):
    # Regla de negocio mezclada con infraestructura
    if monto <= 0:
        raise ValueError('Monto inválido')  # Regla de negocio
    # Acceso directo a DB desde el controlador HTTP
    with engine.connect() as conn:
        conn.execute('INSERT INTO pagos ...')  # Infraestructura
    # Notificación mezclada con la transacción
    requests.post('https://notify.io/...')  # Adaptador externo
    return {'status': 'ok'}
```

Este código, aunque funcional en su etapa inicial, genera las siguientes consecuencias a mediano y largo plazo:

- **Fragilidad ante el cambio:** Modificar la regla 'monto <= 0' obliga a tocar el mismo archivo que maneja HTTP y SQL, incrementando el riesgo de regresiones.
- **Imposibilidad de pruebas unitarias:** Para probar la validación del monto se necesita una base de datos real y un servidor HTTP activo.
- **Acoplamiento tecnológico:** Migrar de PostgreSQL a MongoDB implica reescribir código que contiene reglas de negocio críticas.
- **Deuda técnica exponencial:** A medida que el sistema crece, cada nueva funcionalidad agrega más dependencias cruzadas, degradando la mantenibilidad.

1.2 ¿Por qué Arquitectura Hexagonal?

La Arquitectura Hexagonal, también conocida como Ports & Adapters (Puertos y Adaptadores), fue formulada por Alistair Cockburn en 2005. Su premisa fundamental es:

Idea en una frase (lenguaje sencillo): el corazón del sistema (dominio + casos de uso) no debe depender de la forma como llegan las solicitudes (HTTP, CLI, eventos) ni de cómo se guardan los

datos (PostgreSQL, MongoDB, archivos). Eso se logra usando 'puertos' (interfaces) y 'adaptadores' (implementaciones).

Principio Fundacional

"Permite que una aplicación sea dirigida igualmente por usuarios, programas, pruebas automatizadas o scripts batch, y que sea desarrollada y probada de forma aislada de sus eventuales dispositivos de ejecución y bases de datos."

— Alistair Cockburn, 2005

Es un subconjunto de la filosofía 'Clean Architecture' de Robert C. Martin, con una topología específica basada en la metáfora del hexágono. Sus objetivos estratégicos son:

1. **Mantenibilidad:** El código de negocio es estable y evoluciona independientemente de la infraestructura.
2. **Modularidad:** Componentes perfectamente delimitados que pueden ser reemplazados sin efecto dominó.
3. **Testabilidad:** El núcleo puede ser probado en milisegundos sin base de datos, sin HTTP, sin servicios externos.
4. **Independencia tecnológica:** La aplicación no queda 'casada' con ningún framework, ORM o base de datos específica.

⚠ La Regla de Oro

Si la implementación de Arquitectura Hexagonal no resulta en un código más limpio, más modular y más fácil de probar, la implementación es incorrecta.

Esta regla será el criterio primario de evaluación en el trabajo extra clase.

2. Anatomía de la Arquitectura: Diseccionando Cada Capa

La metáfora del hexágono no impone exactamente seis lados; es una representación visual de un núcleo rodeado de interfaces. La arquitectura se compone de tres anillos concéntricos: el Dominio en el centro, la Aplicación en el intermedio, y los Adaptadores en el exterior. El siguiente diagrama textual ilustra la estructura:



2.1 El Dominio: El Núcleo Soberano

Definición y responsabilidad: El dominio es el corazón de la aplicación. Contiene el conocimiento que pertenece exclusivamente al negocio: sus reglas, sus invariantes y sus conceptos. Es completamente agnóstico al exterior: desconoce la existencia de bases de datos, frameworks web, servicios externos, colas de mensajes o cualquier detalle técnico.

Composición del dominio:

- **Entidades (Entities):** Objetos con identidad propia que persisten en el tiempo. Ejemplo: Cliente, Transaccion, Pedido. Encapsulan estado y comportamiento de negocio.
- **Objetos de Valor (Value Objects):** Objetos inmutables definidos por sus atributos, sin identidad propia. Ejemplo: Dinero(monto=100, moneda='COP'), Direccion, Email.

- **Agregados (Aggregates):** Cluster de entidades y objetos de valor tratados como una unidad transaccional. Ejemplo: un Pedido con sus ItemsDePedido.
- **Eventos de Dominio (Domain Events):** Registros inmutables de algo que ocurrió. Ejemplo: PagoAprobado, ClienteCreado. Son cruciales para arquitecturas orientadas a eventos.
- **Excepciones de Dominio:** Errores que representan violaciones de reglas de negocio. Ejemplo: SaldoInsuficienteError, LimiteTransaccionExcedidoError.

Implementación en Python con Pydantic:

La elección de Pydantic para el dominio es estratégica: permite definir modelos con validación de tipos estricta sin introducir dependencias de infraestructura como SQLAlchemy o Django ORM.

```
# dominio/entidades.py
from pydantic import BaseModel, field_validator, model_validator
from uuid import UUID, uuid4
from decimal import Decimal
from datetime import datetime
from enum import Enum

class EstadoTransaccion(Enum):
    PENDIENTE = 'PENDIENTE'
    APROBADA = 'APROBADA'
    RECHAZADA = 'RECHAZADA'

class Dinero(BaseModel): # Objeto de Valor
    monto: Decimal
    moneda: str = 'COP'

    @field_validator('monto')
    @classmethod
    def monto_positivo(cls, v):
        if v <= 0:
            raise ValueError('El monto debe ser mayor a cero')
        return v

    def __add__(self, otro: 'Dinero') -> 'Dinero':
        if self.moneda != otro.moneda:
            raise ValueError('No se pueden sumar monedas distintas')
        return Dinero(monto=self.monto + otro.monto, moneda=self.moneda)

class Transaccion(BaseModel): # Entidad
    id: UUID = uuid4()
    cliente_id: UUID
    valor: Dinero
    estado: EstadoTransaccion = EstadoTransaccion.PENDIENTE
    creada_en: datetime = datetime.utcnow()

    def aprobar(self) -> None:
        '''Regla de negocio: solo se puede aprobar si está pendiente'''
        if self.estado != EstadoTransaccion.PENDIENTE:
```

```
        raise ValueError(f'No se puede aprobar una transacción en estado
{self.estado}')
        self.estado = EstadoTransaccion.APROBADA

def rechazar(self) -> None:
    if self.estado != EstadoTransaccion.PENDIENTE:
        raise ValueError('Solo se pueden rechazar transacciones pendientes')
    self.estado = EstadoTransaccion.RECHAZADA
```

Error Crítico — Violación del Dominio

NUNCA incluya imports de librerías de infraestructura en el dominio:

```
from sqlalchemy import Column # ❌ VIOLA la arquitectura
from fastapi import Request   # ❌ VIOLA la arquitectura
import requests               # ❌ VIOLA la arquitectura
```

El único import externo permitido en el dominio es Pydantic y las librerías estándar de Python.

2.2 Los Puertos: El Contrato entre el Núcleo y el Exterior

Definición: Los puertos son interfaces abstractas —contratos estrictos— que definen cómo el mundo exterior puede interactuar con el dominio y cómo el dominio puede solicitar servicios al exterior. No son clases concretas; son especificaciones de comportamiento.

En Python se implementan mediante dos mecanismos:

- `abc.ABC` con `@abstractmethod`: Para contratos formales con herencia explícita.
- `typing.Protocol`: Para contratos estructurales (duck typing). Más flexible y no requiere herencia.

2.2.1 Puertos de Entrada (Input Ports)

Propósito: Definen cómo el mundo exterior "habla" con la aplicación. Son la "Definición de Uso": especifican qué acciones están disponibles para los actores externos.

```
# puertos/entrada.py
from abc import ABC, abstractmethod
from uuid import UUID
from dominio.entidades import Transaccion
from dataclasses import dataclass

@dataclass
class ComandoCrearTransaccion:
    '''DTO: Data Transfer Object - lleva datos sin lógica'''
    cliente_id: UUID
    monto: float
    moneda: str
```

```
class ServicioTransaccionesPort(ABC):
    '''Puerto de Entrada: Contrato de uso para el mundo exterior'''

    @abstractmethod
    def crear_transaccion(self, comando: ComandoCrearTransaccion) -> Transaccion:
        '''Crea y registra una nueva transacción de pago'''
        ...

    @abstractmethod
    def aprobar_transaccion(self, transaccion_id: UUID) -> Transaccion:
        '''Aprueba una transacción pendiente'''
        ...

    @abstractmethod
    def consultar_transaccion(self, transaccion_id: UUID) -> Transaccion:
        '''Recupera el estado de una transacción por su ID'''
        ...
```

2.2.2 Puertos de Salida (Output Ports)

Propósito: Definen lo que el dominio "necesita" del exterior para operar. Son la "Definición de Necesidad": el caso de uso los invoca sin saber quién los implementa.

```
# puertos/salida.py
from abc import ABC, abstractmethod
from uuid import UUID
from typing import Optional
from dominio.entidades import Transaccion, Cliente

class TransaccionRepositorioPort(ABC):
    '''Puerto de Salida: Contrato de persistencia (no menciona PostgreSQL ni MongoDB)'''

    @abstractmethod
    def guardar(self, transaccion: Transaccion) -> None: ...

    @abstractmethod
    def buscar_por_id(self, transaccion_id: UUID) -> Optional[Transaccion]: ...

    @abstractmethod
    def listar_por_cliente(self, cliente_id: UUID) -> list[Transaccion]: ...

class NotificadorPort(ABC):
    '''Puerto de Salida: Contrato de notificaciones (no menciona email ni SMS)'''

    @abstractmethod
    def notificar_pago_aprobado(self, transaccion: Transaccion) -> None: ...

    @abstractmethod
    def notificar_pago_rechazado(self, transaccion: Transaccion) -> None: ...
```

Regla Crítica de Nomenclatura

Los puertos se nombran por **INTENCIÓN** de negocio, **NO** por tecnología:

Puerto	Evaluación
TransaccionRepositorioPort	✓ Correcto — expresa la intención
PostgresRepository	✗ Incorrecto — revela la tecnología
NotificadorPort	✓ Correcto — expresa la intención
KafkaPublisher	✗ Incorrecto — expresa la tecnología
ClienteQueryPort	✓ Correcto — expresa la intención
MongoDBAdapter	✗ Incorrecto — revela la implementación

2.3 La Capa de Aplicación: Los Casos de Uso

Definición: La capa de aplicación contiene los Casos de Uso (Use Cases): orquestadores que coordinan el flujo entre el Puerto de Entrada, las entidades del Dominio y el Puerto de Salida. Preferimos el término 'Caso de Uso' sobre 'Servicio' porque expresa la intención del negocio, no un rol técnico.

Responsabilidades del Caso de Uso:

5. Recibir el comando/query desde el Puerto de Entrada.
6. Validar pre-condiciones de orquestación (no de dominio).
7. Invocar las reglas de negocio en las entidades del Dominio.
8. Utilizar los Puertos de Salida para persistir o comunicar.
9. Retornar el resultado al Puerto de Entrada.

```
# aplicacion/casos_de_uso.py
from puertos.entrada import ServicioTransaccionesPort, ComandoCrearTransaccion
from puertos.salida import TransaccionRepositorioPort, NotificadorPort
from dominio.entidades import Transaccion, Dinero
from uuid import UUID
from decimal import Decimal

class ServicioTransacciones(ServicioTransaccionesPort):
    '''Caso de Uso: implementa el Puerto de Entrada,
       consume los Puertos de Salida mediante inyección de dependencias'''

    def __init__(
        self,
```

```
        repositorio: TransaccionRepositorioPort, # Inyección — no sabe si es PG o
Mongo
        notificador: NotificadorPort           # Inyección — no sabe si es
email o SMS
    ):
        self._repositorio = repositorio
        self._notificador = notificador

    def crear_transaccion(self, comando: ComandoCrearTransaccion) -> Transaccion:
        # 1. Construir entidad del dominio (validación de dominio ocurre aquí)
        valor = Dinero(monto=Decimal(str(comando.monto)), moneda=comando.moneda)
        transaccion = Transaccion(cliente_id=comando.cliente_id, valor=valor)

        # 2. Persistir a través del puerto de salida
        self._repositorio.guardar(transaccion)

        return transaccion

    def aprobar_transaccion(self, transaccion_id: UUID) -> Transaccion:
        # 1. Recuperar entidad
        transaccion = self._repositorio.buscar_por_id(transaccion_id)
        if transaccion is None:
            raise ValueError(f'Transacción {transaccion_id} no encontrada')

        # 2. Invocar regla de negocio en la ENTIDAD (no aquí)
        transaccion.aprobar()

        # 3. Persistir cambio
        self._repositorio.guardar(transaccion)

        # 4. Notificar (a través del puerto de salida)
        self._notificador.notificar_pago_aprobado(transaccion)

        return transaccion

    def consultar_transaccion(self, transaccion_id: UUID) -> Transaccion:
        transaccion = self._repositorio.buscar_por_id(transaccion_id)
        if transaccion is None:
            raise ValueError(f'Transacción {transaccion_id} no encontrada')
        return transaccion
```

2.4 Los Adaptadores: La Conexión con el Mundo Real

Definición: Los adaptadores son las implementaciones concretas de los puertos. Son el único lugar donde se mencionan tecnologías específicas como FastAPI, SQLAlchemy, Kafka o AWS S3. Existen dos tipos:

2.4.1 Adaptadores de Entrada (Driving / Primary Adapters)

Son los actores que inician la interacción con la aplicación. Su único trabajo es traducir el mecanismo externo (HTTP, CLI, evento) al lenguaje del Puerto de Entrada.

```
# adaptadores/entrada/api_rest.py
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from uuid import UUID
from puertos.entrada import ServicioTransaccionesPort, ComandoCrearTransaccion

router = APIRouter(prefix='/transacciones', tags=['Transacciones'])

class CrearTransaccionRequest(BaseModel): # DTO de entrada HTTP
    cliente_id: UUID
    monto: float
    moneda: str = 'COP'

class TransaccionResponse(BaseModel): # DTO de salida HTTP
    id: str
    estado: str
    monto: float

def get_servicio() -> ServicioTransaccionesPort:
    from config.contenedor import contenedor # Inyección de dependencias
    return contenedor.servicio_transacciones

@router.post('/', response_model=TransaccionResponse)
def crear_transaccion(
    request: CrearTransaccionRequest,
    servicio: ServicioTransaccionesPort = Depends(get_servicio)
):
    '''El controlador NO tiene lógica de negocio. Solo traduce HTTP → Puerto'''
    comando = ComandoCrearTransaccion(
        cliente_id=request.cliente_id,
        monto=request.monto,
        moneda=request.moneda
    )
    try:
        transaccion = servicio.crear_transaccion(comando)
        return TransaccionResponse(
            id=str(transaccion.id),
            estado=transaccion.estado.value,
            monto=float(transaccion.valor.monto)
        )
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

2.4.2 Adaptadores de Salida (Driven / Secondary Adapters)

Son los sistemas que la aplicación 'conduce' o maneja. Implementan los Puertos de Salida con tecnologías concretas.

```
# adaptadores/salida/repositorio_postgres.py
from sqlalchemy import create_engine, Column, String, Numeric, Enum as SAEnum
from sqlalchemy.orm import declarative_base, Session
```

```
from puertos.salida import TransaccionRepositorioPort
from dominio.entidades import Transaccion, Dinero, EstadoTransaccion
from uuid import UUID
from decimal import Decimal

Base = declarative_base()

class TransaccionORM(Base): # Modelo de infraestructura — NO entra al dominio
    __tablename__ = 'transacciones'
    id = Column(String, primary_key=True)
    cliente_id = Column(String, nullable=False)
    monto = Column(Numeric, nullable=False)
    moneda = Column(String, default='COP')
    estado = Column(SAEnum(EstadoTransaccion),
                      default=EstadoTransaccion.PENDIENTE)

class RepositorioTransaccionPostgres(TransaccionRepositorioPort):
    '''Adaptador de Salida: implementa el puerto con PostgreSQL + SQLAlchemy'''

    def __init__(self, session: Session):
        self._session = session

    def guardar(self, transaccion: Transaccion) -> None:
        orm = TransaccionORM(
            id=str(transaccion.id),
            cliente_id=str(transaccion.cliente_id),
            monto=transaccion.valor.monto,
            moneda=transaccion.valor.moneda,
            estado=transaccion.estado
        )
        self._session.merge(orm)
        self._session.commit()

    def buscar_por_id(self, transaccion_id: UUID) -> Transaccion | None:
        orm = self._session.get(TransaccionORM, str(transaccion_id))
        if orm is None:
            return None
        return Transaccion(
            id=UUID(orm.id),
            cliente_id=UUID(orm.cliente_id),
            valor=Dinero(monto=Decimal(str(orm.monto)), moneda=orm.moneda),
            estado=orm.estado
        )
```

2.5 La Regla de Dependencia: El Principio más Importante

Definición: La Regla de Dependencia establece que las dependencias a nivel de código fuente deben apuntar exclusivamente hacia el interior —hacia el dominio. Esta regla es inviolable y es el mecanismo que garantiza el desacoplamiento.

Regla de Dependencia — Resumen Visual

PERMITIDO: Adaptador → Puerto → Caso de Uso → Dominio

PROHIBIDO: Dominio → Puerto (el dominio no conoce los puertos)

PROHIBIDO: Dominio → Adaptador (el dominio nunca conoce la infraestructura)

PROHIBIDO: Caso de Uso → Adaptador concreto (solo conoce el puerto abstracto)

La 'magia' que permite que el flujo de control vaya de izquierda a derecha (FastAPI → Caso de Uso → PostgreSQL) mientras las dependencias de código apuntan hacia el centro, se llama Inversión de Dependencias (Dependency Inversion Principle — DIP). El Caso de Uso depende de la abstracción (el puerto), no de la implementación concreta. En tiempo de ejecución, la implementación concreta se inyecta externamente (contenedor de inyección de dependencias).

```
# config/contenedor.py - Centro de ensamblaje de la aplicación
import os
from aplicacion.casos_de_uso import ServicioTransacciones

def crear_contenedor():
    modo = os.getenv('MODO_REPOSITORIO', 'memoria')

    if modo == 'postgres':
        from adaptadores.salida.repositorio_postgres import
RepositorioTransaccionPostgres
        from sqlalchemy.orm import sessionmaker
        from sqlalchemy import create_engine
        engine = create_engine(os.getenv('DATABASE_URL'))
        session = sessionmaker(bind=engine)()
        repositorio = RepositorioTransaccionPostgres(session)
    else: # modo == 'memoria'
        from adaptadores.salida.repositorio_memoria import
RepositorioTransaccionMemoria
        repositorio = RepositorioTransaccionMemoria()

    from adaptadores.salida.notificador_email import NotificadorEmail
    notificador = NotificadorEmail()

    return ServicioTransacciones(repositorio=repositorio, notificador=notificador)
```

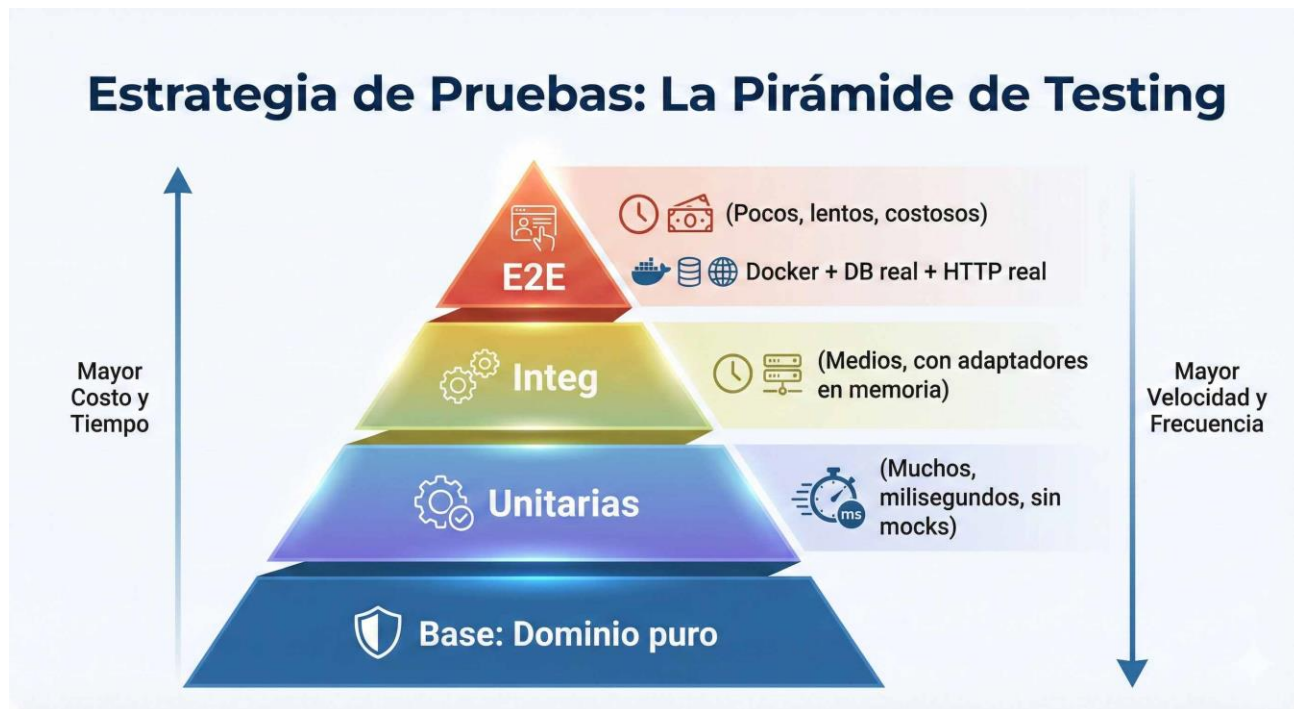
3. Estructura Canónica del Proyecto

La organización de carpetas es la expresión física de la arquitectura. La siguiente estructura ejemplo refleja con precisión los límites entre capas:

```
sistema_pagos/
├── dominio/                                # CAPA 1: Núcleo – CERO deps externas
│   ├── __init__.py
│   ├── entidades.py                       # Transaccion, Cliente, Pedido
│   ├── objetos_valor.py                  # Dinero, Email, Direccion
│   ├── excepciones.py                   # SaldoInsuficienteError, etc.
│   └── eventos.py                       # PagoAprobado, ClienteCreado
├── puertos/                              # CAPA 2: Contratos – solo abstracciones
│   ├── __init__.py
│   ├── entrada.py                       # ServicioTransaccionesPort, DTOs
│   └── salida.py                        # RepositorioPort, NotificadorPort
├── aplicacion/                          # CAPA 3: Orquestación – Casos de Uso
│   ├── __init__.py
│   └── casos_de_uso.py                  # ServicioTransacciones
├── adaptadores/                         # CAPA 4: Infraestructura – detalles técnicos
│   ├── entrada/
│   │   ├── api_rest.py                 # FastAPI Router
│   │   └── cli.py                     # Interfaz de línea de comandos
│   └── salida/
│       ├── repositorio_postgres.py     # SQLAlchemy + PostgreSQL
│       ├── repositorio_mongodb.py      # Motor + MongoDB
│       ├── repositorio_memoria.py      # Dict en memoria (para tests)
│       ├── notificador_email.py        # SMTP / SendGrid
│       └── notificador_sms.py          # Twilio
├── config/
│   ├── contenedor.py                  # Ensamblaje / Inyección de dependencias
│   └── settings.py                   # Variables de entorno
├── tests/
│   ├── unidad/                        # Pruebas del dominio (sin mocks complejos)
│   ├── integracion/                  # Pruebas con adaptadores en memoria
│   └── e2e/                          # Pruebas con infraestructura real (Docker)
├── docker-compose.yml
├── requirements.txt
└── main.py                            # Punto de entrada de la aplicación
```

4. Estrategia de Pruebas: La Pirámide de Testing

Una de las mayores ventajas de la Arquitectura Hexagonal es que habilita una pirámide de pruebas eficiente: muchas pruebas unitarias rápidas, pocas pruebas de integración medianas, y mínimas pruebas end-to-end costosas.



La Arquitectura Hexagonal hace posible la Pirámide de Testing porque aísla el código que cambia con frecuencia y es fácil de probar (el Dominio/Centro) del código que es rígido, lento y difícil de probar (la Infraestructura/Bordes).

A continuación la explicación de cómo se mapea cada nivel de la pirámide con las capas del hexágono:

1. La Base: Pruebas Unitarias y el Dominio Puro

- **En la Pirámide:** Representa la base amplia. Son pruebas muy rápidas (milisegundos), masivas en cantidad y, preferiblemente, sin *mocks* (simulaciones).
- **En el Hexágono:** Corresponde a la **Capa de Dominio** (Entidades, Objetos de Valor, Reglas de negocio) y gran parte de la **Capa de Aplicación** (Casos de Uso).
- **Por qué encajan:** Como la regla de oro de la Arquitectura Hexagonal dicta que el Dominio tiene **cero dependencias externas**, puedes instanciar tus clases de dominio y probar toda tu lógica de negocio en memoria pura. No necesitas levantar una base de datos ni un servidor web. Si un Caso de Uso necesita interactuar con el exterior, simplemente le inyectas una implementación "Falsa" (Fake) o un *Mock* del **Puerto de Salida**, manteniendo la prueba ultrarrápida.

2. El Medio: Pruebas de Integración y los Adaptadores

- **En la Pirámide:** Representa el nivel intermedio. Son menos pruebas que las unitarias, toman un poco más de tiempo y suelen usar adaptadores en memoria (como bases de datos H2 o SQLite) o contenedores ligeros.
- **En el Hexágono:** Corresponde a la **Capa de Infraestructura**, específicamente probando los **Adaptadores** de forma aislada para asegurar que implementan correctamente los Puertos.
- **Por qué encajan:** Aquí es donde verificas que tu código de SQLAlchemy realmente guarda y recupera datos de la base de datos, o que tu router de FastAPI responde con los códigos HTTP correctos. Pruebas que el exterior del hexágono se comunica bien con tus interfaces (Puertos), sin necesidad de probar toda la lógica de negocio (que ya está cubierta en la base).

3. La Cúspide: Pruebas E2E (End-to-End) y el Sistema Completo

- **En la Pirámide:** Es la punta. Son muy pocas pruebas porque son lentas, frágiles y costosas de mantener. Requieren levantar el entorno completo (Docker, DB real, HTTP).
- **En el Hexágono:** Es una petición que atraviesa **absolutamente todas las capas**: Entra por el Adaptador *Driving* (FastAPI), cruza el Puerto de Entrada, ejecuta el Caso de Uso (Aplicación), altera el Dominio, sale por el Puerto de Salida y persiste en el Adaptador *Driven* (SQLAlchemy/PostgreSQL).
- **Por qué encajan:** En la Arquitectura Hexagonal, el E2E se usa solo como una prueba de humo (*smoke test*) para garantizar que el ensamblaje de todas las piezas (la inyección de dependencias y la configuración del framework) funciona correctamente en un entorno similar a producción. No se usan para probar reglas de negocio.

4.1 Pruebas Unitarias del Dominio

El dominio no tiene dependencias externas, por lo que se puede probar instanciando clases Python directamente. No se necesita pytest-mock, base de datos ni servidor.

```
# tests/unidad/test_transaccion.py
import pytest
from uuid import uuid4
from decimal import Decimal
from dominio.entidades import Transaccion, Dinero, EstadoTransaccion

def test_crear_transaccion_estado_inicial_es_pendiente():
    t = Transaccion(cliente_id=uuid4(), valor=Dinero(monto=Decimal('100.00')))
    assert t.estado == EstadoTransaccion.PENDIENTE

def test_aprobar_transaccion_cambia_estado_a_aprobada():
    t = Transaccion(cliente_id=uuid4(), valor=Dinero(monto=Decimal('50.00')))
    t.aprobar()
    assert t.estado == EstadoTransaccion.APROBADA

def test_aprobar_transaccion_ya_aprobada_lanza_excepcion():
    t = Transaccion(cliente_id=uuid4(), valor=Dinero(monto=Decimal('50.00')))
    t.aprobar()
    with pytest.raises(ValueError, match='No se puede aprobar'):
        t.aprobar() # Segunda aprobación debe fallar
```

```
def test_dinero_monto_negativo_lanza_validacion_error():
    with pytest.raises(Exception): # Pydantic ValidationError
        Dinero(monto=Decimal('-1.00'))

def test_suma_dinero_misma_moneda():
    a = Dinero(monto=Decimal('100.00'), moneda='COP')
    b = Dinero(monto=Decimal('50.00'), moneda='COP')
    resultado = a + b
    assert resultado.monto == Decimal('150.00')
```

4.2 Pruebas de Integración con Adaptadores en Memoria

Para probar los Casos de Uso, se reemplazan los adaptadores reales con implementaciones en memoria (Fakes). Esto permite probar la orquestación completa sin infraestructura.

```
# adaptadores/salida/repositorio_memoria.py - Fake Adapter
from puertos.salida import TransaccionRepositorioPort, NotificadorPort
from dominio.entidades import Transaccion
from uuid import UUID

class RepositorioTransaccionMemoria(TransaccionRepositorioPort):
    def __init__(self): self._store: dict[UUID, Transaccion] = {}
    def guardar(self, t: Transaccion) -> None: self._store[t.id] = t
    def buscar_por_id(self, id: UUID): return self._store.get(id)
    def listar_por_cliente(self, cid: UUID): return [t for t in
self._store.values() if t.cliente_id == cid]

class NotificadorMemoria(NotificadorPort):
    def __init__(self): self.notificaciones_enviadas = []
    def notificar_pago_aprobado(self, t):
self.notificaciones_enviadas.append(('aprobado', t.id))
    def notificar_pago_rechazado(self, t):
self.notificaciones_enviadas.append(('rechazado', t.id))

# tests/integracion/test_caso_uso_transacciones.py
def test_crear_y_aprobar_transaccion_notifica_correctamente():
    repo = RepositorioTransaccionMemoria()
    notificador = NotificadorMemoria()
    servicio = ServicioTransacciones(repositorio=repo, notificador=notificador)

    comando = ComandoCrearTransaccion(cliente_id=uuid4(), monto=200.0,
moneda='COP')
    transaccion = servicio.crear_transaccion(comando)
    servicio.aprobar_transaccion(transaccion.id)

    assert ('aprobado', transaccion.id) in notificador.notificaciones_enviadas
    guardada = repo.buscar_por_id(transaccion.id)
    assert guardada.estado.value == 'APROBADA'
```

5. Evolución de la Aplicación Apoyada en la Arquitectura Hexagonal

Cómo evolucionar una aplicación con arquitectura hexagonal (guía práctica):

- 1) Cambios por 'canal' (entradas): si aparece un nuevo canal (WhatsApp, CLI, batch, eventos), crea un adaptador de entrada nuevo. El núcleo no cambia.
- 2) Cambios por 'tecnología' (salidas): si cambia la base de datos, el proveedor de correo o un API externo, crea un adaptador de salida nuevo que implemente el mismo puerto.
- 3) Cambios por 'regla de negocio' (dominio): si cambia una regla (por ejemplo, la multa, límites, estados), cambia solo el dominio y ajusta pruebas unitarias; los adaptadores deberían requerir pocos o cero cambios.
- 4) Cambios por 'flujo' (casos de uso): si aparece un caso de uso nuevo (por ejemplo 'reservar libro'), normalmente agregas un puerto de entrada + un caso de uso; el dominio puede reutilizar entidades existentes.

Tip para estudiantes: cuando hagas un cambio, intenta responder: ¿este cambio es de negocio, de canal o de tecnología? Esa respuesta te dice en qué carpeta debería vivir el cambio.

Una de las promesas más poderosas de la Arquitectura Hexagonal es que el sistema puede crecer, escalar y transformarse radicalmente sin tocar el núcleo de negocio. A continuación se detallan los escenarios de evolución más comunes:

5.1 Evolución 1: Agregar un Nuevo Canal de Entrada

Escenario: El sistema de pagos inició con una API REST. Ahora se requiere también una interfaz de línea de comandos (CLI) para operaciones administrativas, y un consumidor de colas RabbitMQ para procesamiento asíncrono.

Impacto en la arquitectura: Solo se crean nuevos Adaptadores de Entrada. El Dominio, los Puertos y los Casos de Uso permanecen INTACTOS al 100%.

```
# adaptadores/entrada/cli.py - NUEVO adaptador, cero cambios al núcleo
import click
from uuid import UUID
from config.contenedor import crear_contenedor
from puertos.entrada import ComandoCrearTransaccion

@click.command()
@click.option('--cliente-id', required=True, type=str)
@click.option('--monto', required=True, type=float)
def aprobar_pago(cliente_id: str, monto: float):
    servicio = crear_contenedor()
    comando = ComandoCrearTransaccion(
        cliente_id=UUID(cliente_id), monto=monto, moneda='COP'
    )
    t = servicio.crear_transaccion(comando)
    servicio.aprobar_transaccion(t.id)
    click.echo(f'Pago {t.id} aprobado exitosamente')

# adaptadores/entrada/consumidor_rabbitmq.py - NUEVO adaptador
import pika, json
```

```
from config.contenedor import crear_contenedor
from puertos.entrada import ComandoCrearTransaccion
from uuid import UUID

def procesar_mensaje(ch, method, properties, body):
    '''Traduce mensaje RabbitMQ → llamado al Puerto de Entrada'''
    datos = json.loads(body)
    servicio = crear_contenedor()
    comando = ComandoCrearTransaccion(
        cliente_id=UUID(datos['cliente_id']),
        monto=datos['monto'],
        moneda=datos.get('moneda', 'COP')
    )
    servicio.crear_transaccion(comando)
    ch.basic_ack(delivery_tag=method.delivery_tag)
```

5.2 Evolución 2: Migración de Base de Datos (PostgreSQL → MongoDB)

Escenario: El volumen de transacciones creció exponencialmente. El equipo decide migrar de PostgreSQL a MongoDB para mayor escalabilidad en lecturas.

Impacto en la arquitectura: Solo se escribe un nuevo Adaptador de Salida que implementa el mismo Puerto. El Dominio, los Casos de Uso y los Adaptadores de Entrada no se tocan.

```
# adaptadores/salida/repositorio_mongodb.py - NUEVO adaptador
from pymongo import MongoClient
from puertos.salida import TransaccionRepositorioPort
from dominio.entidades import Transaccion, Dinero, EstadoTransaccion
from uuid import UUID
from decimal import Decimal

class RepositorioTransaccionMongoDB(TransaccionRepositorioPort):
    def __init__(self, uri: str, db: str):
        self._col = MongoClient(uri)[db]['transacciones']

    def guardar(self, t: Transaccion) -> None:
        self._col.update_one(
            {'_id': str(t.id)},
            {'$set': {'cliente_id': str(t.cliente_id), 'monto':
float(t.valor.monto),
                'moneda': t.valor.moneda, 'estado': t.estado.value}},
            upsert=True
        )

    def buscar_por_id(self, id: UUID) -> Transaccion | None:
        doc = self._col.find_one({'_id': str(id)})
        if not doc: return None
        return Transaccion(
            id=UUID(doc['_id']),
            cliente_id=UUID(doc['cliente_id']),
            valor=Dinero(monto=Decimal(str(doc['monto'])), moneda=doc['moneda']),
            estado=EstadoTransaccion(doc['estado'])
        )
```

```
# config/contenedor.py — Solo cambia 1 línea aquí
# De: repositorio = RepositorioTransaccionPostgres(session)
# A:  repositorio = RepositorioTransaccionMongoDB(MONGO_URI, 'pagos')
```

5.3 Evolución 3: Transición a Arquitectura de Microservicios

Escenario: La empresa crece y decide separar el sistema monolítico en microservicios. El módulo de pagos se convierte en su propio servicio independiente.

La Arquitectura Hexagonal facilita enormemente esta transición porque las fronteras ya están definidas. Cada futuro microservicio ya existe como un conjunto coherente de capas. El proceso es:

10. **Identificar el Contexto Delimitado (Bounded Context):** Pagos, Clientes, Notificaciones son candidatos naturales.
11. **Extraer el módulo:** La carpeta dominio/, puertos/, aplicacion/ y adaptadores/ del contexto se mueven a un nuevo repositorio.
12. **Agregar comunicación entre servicios:** Se crea un nuevo Adaptador de Salida que publica eventos en un Message Broker (Kafka, RabbitMQ) en lugar de llamar directamente al otro servicio.

```
# Microservicio de Pagos — Adaptador de Salida para comunicación inter-servicio
# adaptadores/salida/publicador_kafka.py
import json
from kafka import KafkaProducer
from puertos.salida import NotificadorPort
from dominio.entidades import Transaccion
from dominio.eventos import PagoAprobado

class PublicadorEventosKafka(NotificadorPort):
    '''El Dominio sigue igual. Solo el ADAPTADOR conoce Kafka'''
    def __init__(self, bootstrap_servers: str):
        self._producer = KafkaProducer(bootstrap_servers=bootstrap_servers,
                                       value_serializer=lambda v:
                                       json.dumps(v).encode('utf-8'))

    def notificar_pago_aprobado(self, t: Transaccion) -> None:
        evento = PagoAprobado(transaccion_id=t.id, cliente_id=t.cliente_id,
                              monto=float(t.valor.monto))
        self._producer.send('pagos.aprobados', evento.dict())
        self._producer.flush()
```

5.4 Evolución 4: Integración con Sistemas de IoT

Escenario: Un sistema de domótica requiere reaccionar a eventos de sensores IoT (temperatura, movimiento). ¿Dónde se ubicaría el sensor?

Respuesta Arquitectónica

El sensor IoT es un Adaptador de ENTRADA (Driving Adapter). Aunque los datos 'fluyen desde afuera hacia adentro', la clasificación se basa en quién INICIA la interacción. El sensor es quien dispara el evento hacia la aplicación, por tanto es un actor primario (Driving).

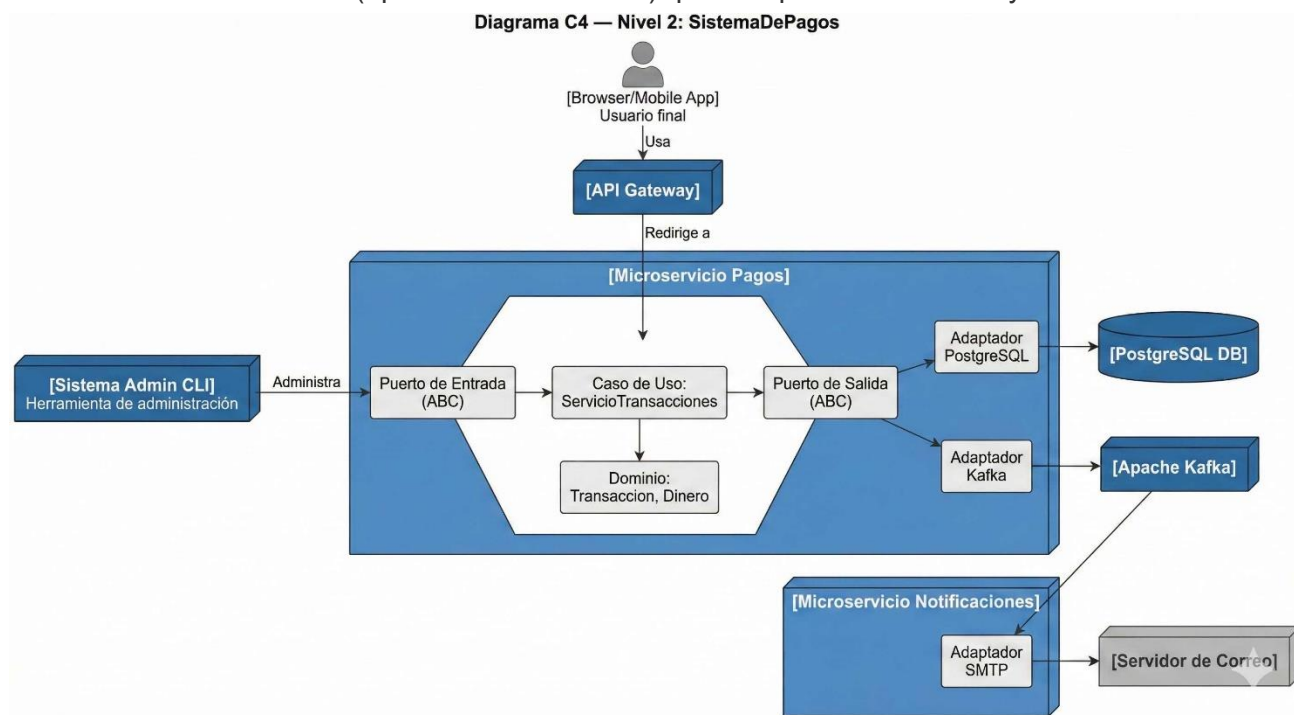
La solución: crear un adaptador MQTT o WebSocket que escuche los mensajes del sensor y los traduzca al lenguaje del Puerto de Entrada. El dominio (reglas como 'si temperatura > 30°C encender ventilador') permanece completamente puro.

6. Diagramación Formal Requerida

Para toda documentación arquitectónica se exigen los siguientes diagramas. Se recomienda el modelo C4 o el estándar UML con mapeo explícito de componentes hexagonales.

6.1 Modelo C4 — Nivel 2: Diagrama de Contenedores

Muestra los contenedores (aplicaciones/servicios) que componen el sistema y cómo se comunican.



1. La Vista Macro: Modelo C4 (Nivel 2 - Contenedores)

En el Nivel 2 de C4, el objetivo es mostrar las unidades desplegables independientes y cómo se comunican entre sí. En este esquema, los contenedores son los bloques azules y los repositorios de datos/mensajería:

- **Aplicaciones y Gateways:** El [Browser/Mobile App] (Usuario final) interactúa con el [API Gateway], que actúa como el punto de entrada único y enrutador hacia los servicios internos. También existe una herramienta de administración directa, el [Sistema Admin CLI].
- **Microservicios (Lógica de Negocio):** Tenemos dos contenedores principales: [Microservicio Pagos] y [Microservicio Notificaciones]. Son unidades que pueden escalarse y desplegarse de forma independiente.
- **Infraestructura (Bases de datos y Mensajería):** Los contenedores externos que almacenan y transmiten estado: [PostgreSQL DB], el bus de eventos [Apache Kafka], y un servicio externo [Servidor de Correo].

2. La Vista Micro: Arquitectura Hexagonal (Dentro del Microservicio)

Lo más valioso del diagrama es cómo hace un "zoom" dentro del [Microservicio Pagos] para revelar su diseño hexagonal, demostrando el principio de Inversión de Dependencias.

- **El Lado *Driving* (Izquierda - Entrada):**

- Los actores externos (API Gateway y Sistema Admin CLI) quieren ejecutar acciones en el sistema.
- No tocan la lógica directamente, sino que se comunican a través del **Puerto de Entrada (ABC)**. Este puerto define *qué* puede hacer la aplicación (las interfaces). En la práctica, aquí estarían los controladores (ej. routers de FastAPI o comandos de Click) que adaptan la petición HTTP o de consola a un formato que el caso de uso entienda.
- **El Centro (El Hexágono / Núcleo):**
 - **Caso de Uso (ServicioTransacciones):** Es el orquestador de la aplicación. Recibe la petición del puerto de entrada, aplica las reglas de negocio y coordina el trabajo.
 - **Dominio (Transaccion, Dinero):** Es el corazón del software. Contiene las Entidades y Objetos de Valor. Como dicta la regla estricta: **no tiene dependencias externas**. No sabe nada de HTTP, bases de datos o Kafka.
- **El Lado Driven (Derecha - Salida):**
 - Cuando el Caso de Uso necesita persistir datos o avisar a otros servicios, no llama a PostgreSQL o Kafka directamente. Utiliza el **Puerto de Salida (ABC)**, que es una interfaz abstracta (ej. TransaccionRepository).
 - Los **Adaptadores** (Adaptador PostgreSQL y Adaptador Kafka) son los que *implementan* esas interfaces. Ellos contienen la lógica de infraestructura (ej. consultas de SQLAlchemy o clientes de publicación de Kafka) para hablar con el mundo exterior.

3. El Flujo de Eventos Inter-Contenedor

El diagrama también muestra cómo la arquitectura hexagonal facilita la comunicación asíncrona entre microservicios:

1. El Microservicio Pagos finaliza una transacción y usa su Adaptador Kafka para publicar un evento de dominio (ej. PagoCompletado).
2. Apache Kafka transporta el mensaje.
3. El [Microservicio Notificaciones] actúa como consumidor (tendría su propio puerto de entrada impulsado por Kafka). Al procesar el evento, utiliza su **Adaptador SMTP** (un adaptador de salida) para enviar el recibo mediante el [Servidor de Correo].

En resumen, el diagrama ilustra de manera impecable cómo proteger el dominio (Transaccion, Dinero) de las tecnologías volátiles (PostgreSQL, Kafka, APIs HTTP), asegurando que las dependencias fluyan siempre hacia el centro.

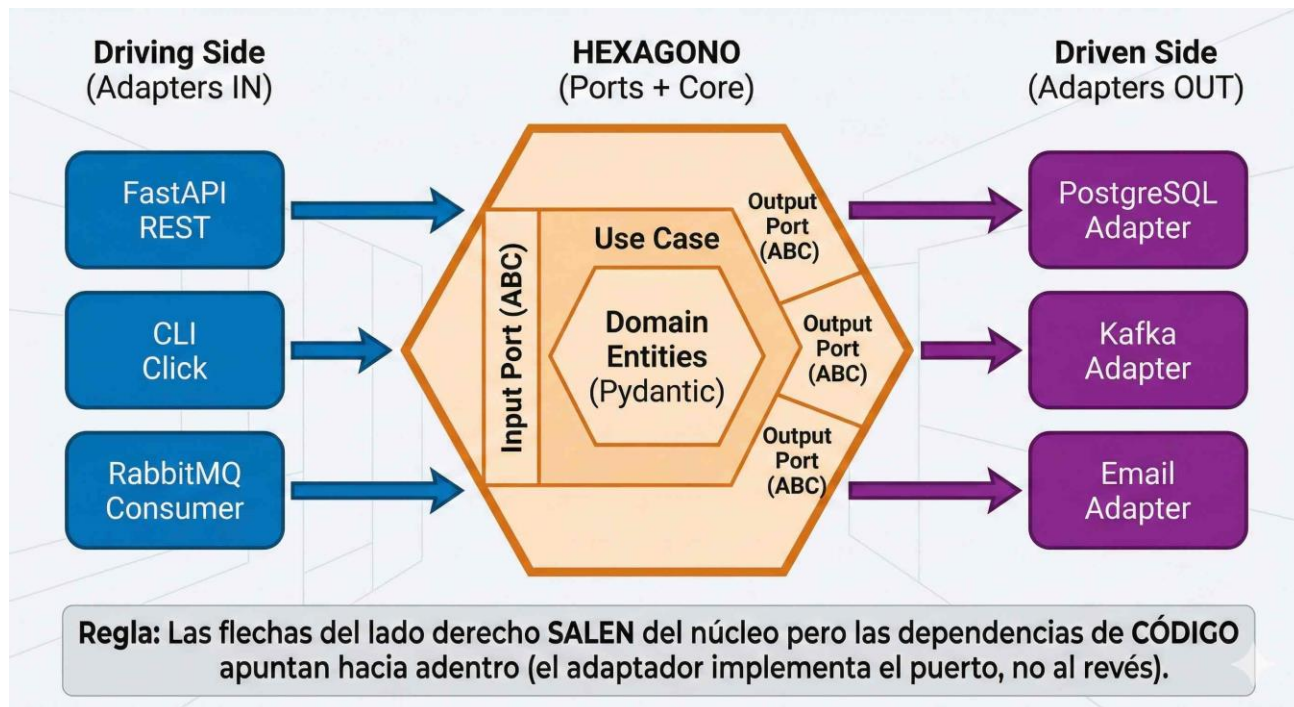
Nota El **Modelo C4** (creado por Simon Brown) es altamente recomendado hoy en día porque proporciona un enfoque jerárquico similar a un mapa de Google Maps, permitiendo hacer "zoom" en la arquitectura a través de cuatro niveles:

- **Contexto (Nivel 1):** El panorama general (usuarios y sistemas externos).
- **Contenedores (Nivel 2):** Las aplicaciones desplegables y bases de datos (como el diagrama que analizamos anteriormente).
- **Componentes (Nivel 3):** Los bloques de construcción internos dentro de un contenedor.
- **Código (Nivel 4):** Las clases o interfaces reales (a menudo opcional si el código es limpio).

Se recomienda porque es mucho más intuitivo y ágil que el UML tradicional para representar sistemas distribuidos y microservicios, hablando un lenguaje que tanto el negocio como los desarrolladores entienden.

6.2 Diagrama de Puertos y Adaptadores (Vista Hexagonal)

Diagrama obligatorio que muestra explícitamente la estructura hexagonal del componente principal:



1. Driving Side (Lado Izquierdo - Adaptadores de Entrada)

Esta es la zona que "conduce" o inicia las acciones en la aplicación.

- **Adaptadores (Adapters IN):** Son los puntos de contacto con el mundo exterior. El diagrama muestra que un sistema puede ser impulsado por múltiples mecanismos simultáneamente: peticiones HTTP vía **FastAPI REST**, comandos de terminal mediante **CLI Click**, o eventos asíncronos consumidos a través de **RabbitMQ Consumer**.
- **Su función:** Traducir los datos específicos de su tecnología (un JSON HTTP, un argumento de consola, o un payload de RabbitMQ) a una estructura limpia y pasarlos al hexágono a través de un Puerto de Entrada.

2. El Hexágono (Centro - Puertos y Núcleo)

Es el corazón inmutable del software, completamente aislado de frameworks externos.

- **Input Port (Puerto de Entrada):** Definido típicamente en Python usando Clases Base Abstractas (ABC), expone las operaciones que el sistema *puede* realizar, actuando como una barrera protectora.
- **Caso de Uso (Use Case):** Contiene la lógica de aplicación. Orquesta las acciones, recibe comandos de los puertos de entrada, interactúa con el dominio y delega la persistencia o notificaciones a los puertos de salida.

- **Dominio (Domain Entities):** Representa las reglas de negocio puras, modeladas en este caso con **Pydantic** para la validación de estructuras de datos.
- **Output Ports (Puertos de Salida):** Interfaces (ABC) que definen lo que el núcleo *necesita* del mundo exterior (por ejemplo, "guardar una transacción" o "enviar una notificación") sin importar cómo se implemente.

3. Driven Side (Lado Derecho - Adaptadores de Salida)

Esta es la infraestructura que la aplicación "conduce" o controla para realizar efectos secundarios.

- **Adaptadores (Adapters OUT):** Reciben las instrucciones del núcleo (a través de los Output Ports) y las traducen al lenguaje específico de la infraestructura.
- **Ejemplos:** El diagrama ilustra adaptadores para interactuar con una base de datos (**PostgreSQL Adapter**), un bus de eventos (**Kafka Adapter**) y un servicio de correo (**Email Adapter**).

4. La Regla de Oro: Inversión de Dependencias (DIP)

El texto en la parte inferior del diagrama resalta el concepto más crítico para que esta arquitectura funcione: *"Las flechas del lado derecho SALEN del núcleo pero las dependencias de CÓDIGO apuntan hacia adentro (el adaptador implementa el puerto, no al revés)".*

- **Flujo de Control vs. Flujo de Dependencias:** Aunque durante la ejecución del programa el Caso de Uso llama a la base de datos (las flechas horizontales apuntan de izquierda a derecha), a nivel de código fuente, el Caso de Uso solo importa y conoce la interfaz abstracta (Output Port). Es el PostgreSQL Adapter el que debe importar esa interfaz para implementarla, apuntando su dependencia hacia el centro.

Esta separación estricta es lo que permite que el núcleo del sistema sea testeable de forma rápida (mediante pruebas unitarias) sin necesidad de levantar bases de datos ni servicios web, tal como lo vimos previamente en la Pirámide de Testing.

7. Trabajo Extra Clase

Refactorización de un Monolito a Arquitectura Hexagonal con Evolución a Microservicio

Datos del Trabajo	
Asignatura	Ingeniería de Software II
Semestre	Decimo semestre — Ingeniería de Sistemas
Modalidad	Individual o en grupo
Punto de partida	Proyecto propio de los estudiantes
Repositorio	GitHub/GitLab con dos ramas: main (monolito original), hexagonal (refactorizado)

7.1 Descripción General

Este trabajo aborda un escenario real: la mayor parte del trabajo de un ingeniero de software no consiste en diseñar sistemas desde cero, sino en evolucionar sistemas existentes que acumularon deuda técnica. El objetivo es tomar un sistema monolítico acoplado, refactorizarlo aplicando Arquitectura Hexagonal, y demostrar que la nueva arquitectura soporta una evolución que sería imposible (o extremadamente costosa) en el monolito original.

Punto de partida: Proyecto previo de los estudiantes desarrollado como un monolítico de referencia.

7.2 Fases del Trabajo

El trabajo se desarrolla en fases secuenciales:

Fase 1: Análisis del Monolito y Mapa de Deuda Técnica

En esta fase no se programa (o se programa muy poco). La meta es entender el sistema actual y sus problemas.

- **Qué debes lograr:**
 - Identificar dónde está mezclada la lógica de negocio (por ejemplo: validaciones dentro de controladores o consultas SQL en la misma función que decide reglas).
 - Dibujar un mapa simple de dependencias (qué módulo llama a cuál).
 - Proponer el primer borrador de tu 'núcleo': entidades, reglas y puertos.

- **Checklist de salida (Definition of Done):** puedes explicar en voz alta (sin leer) cuál es el problema principal de acoplamiento y cuál sería el primer paso para separarlo.
- **Objetivo:** Comprender el sistema existente y documentar sus problemas arquitectónicos antes de tocar una sola línea de código.
- **Actividades:**
 - Crear un diagrama de dependencias del monolito.
 - Identificar y documentar TODAS las violaciones arquitectónicas.
 - Definir el modelo de dominio y diseñar el catálogo de puertos.
- **Entregable Parcial:** Documento 'Análisis del Monolito' (mínimo 5 páginas) con: diagrama de dependencias del estado actual, lista de violaciones encontradas con número de línea, y el diseño propuesto de la arquitectura hexagonal (aún sin código).

Fase 2: Implementación del Dominio y Puertos

Aquí construyes el corazón del sistema: dominio + puertos. La regla es que esta parte debe poder ejecutarse y probarse sin internet, sin base de datos y sin framework web.

Qué debes lograr:

- Entidades y objetos de valor con reglas claras (métodos que validan invariantes).
- Excepciones de dominio con mensajes comprensibles.
- Puertos de entrada y salida (interfaces) nombrados por intención de negocio.

Pruebas sugeridas:

- 1 prueba por cada regla de negocio importante (empieza por los casos más críticos).

Checklist de salida: al ejecutar pruebas unitarias, todo pasa en segundos y no necesitas levantar PostgreSQL ni FastAPI.

Objetivo: Construir el núcleo del sistema sin ninguna dependencia externa.

Actividades:

- Implementar TODAS las entidades del dominio con Pydantic, con validaciones y reglas de negocio encapsuladas.
- Implementar los Objetos de Valor necesarios.
- Definir las interfaces ABC de los Puertos de Entrada y Salida.
- Escribir el 100% de las pruebas unitarias del dominio (mínimo 20 pruebas).
- Ejecutar el script de verificación de pureza arquitectónica y documentar el resultado (debe dar 0 violaciones).

Criterio de avance: Para pasar a la Fase 3, el estudiante debe demostrar al docente que: (1) el dominio no tiene imports externos prohibidos, y (2) todas las pruebas unitarias pasan.

Fase 3: Implementación de la Capa de Aplicación y Adaptadores de Salida

En esta fase conectas el dominio con el mundo mediante casos de uso (capa de aplicación) y construyes adaptadores de salida (persistencia/notificaciones).

Qué debes lograr:

- Casos de uso que orquestan: recibir comando → cargar entidad → llamar regla de dominio → persistir → devolver resultado.
 - Un repositorio en memoria (Fake) para pruebas rápidas.
 - Un repositorio real (por ejemplo PostgreSQL) como segundo adaptador.
- Checklist de salida: los mismos casos de uso funcionan con el repositorio en memoria y con el repositorio real cambiando solo configuración.

Objetivo: Implementar los Casos de Uso y los Adaptadores de Salida.

Actividades:

- Implementar los Casos de Uso en la capa de aplicación.
- Implementar el Adaptador de Salida en Memoria (para pruebas).
- Implementar el Adaptador de Salida con SQLAlchemy (PostgreSQL).
- Implementar el Adaptador de Salida de Notificaciones (puede ser en memoria para esta fase).
- Escribir las pruebas de integración usando adaptadores en memoria.

Fase 4: Adaptadores de Entrada + Evolución Demostrativa

Ahora agregas canales de entrada (por ejemplo API REST) y demuestras evolución: un cambio que antes era difícil, ahora se vuelve local y controlado.

Qué debes lograr:

- Adaptador de entrada (HTTP/CLI/eventos) que solo traduzca datos hacia el puerto de entrada.
- Al menos una evolución demostrativa: nuevo canal de entrada o nuevo adaptador de salida, sin tocar el dominio.

Consejo: documenta el cambio con un pequeño 'antes/después' (qué archivos tocaste y por qué).
Checklist de salida: puedes mostrar un commit donde agregas un adaptador nuevo y no modificas entidades del dominio.

Objetivo: Conectar el núcleo con el mundo exterior y demostrar la capacidad de evolución.

Actividades:

- **API REST con FastAPI:** Implementar TODOS los endpoints del sistema original, más al menos 2 endpoints nuevos que el monolito original no tenía y que solo son posibles gracias a la arquitectura hexagonal.
- Segundo canal de entrada (RECOMENDADO): Implementar UNO de los siguientes: CLI con Click, Consumidor de Webhooks, Script de carga masiva desde CSV, o Consumidor de colas (RabbitMQ/Redis).
- Nuevo adaptador de salida (RECOMENDADO): Implementar un segundo adaptador de persistencia DIFERENTE al de PostgreSQL (MongoDB, Redis, DynamoDB, o archivo JSON) y demostrar que el sistema funciona con ambos sin cambiar el dominio.

Fase 5: Documentación, Pruebas Finales y Preparación de Presentación

Esta fase es de cierre: orden, evidencia y comunicación técnica.

Qué debes lograr:

- README claro: cómo ejecutar en modo memoria y en modo real.
- Evidencia de pruebas (cobertura o al menos listado de pruebas ejecutadas).
- Diagramas mínimos (C4 o una vista hexagonal) que coincidan con tu código.

Checklist de salida: cualquier compañero puede clonar tu repo, seguir el README y ejecutar el proyecto sin adivinar.

Objetivo: Consolidar evidencia, completar la documentación y preparar la defensa.

Actividades:

- Completar el informe técnico final.
- Asegurarse de que la cobertura de pruebas es $\geq 85\%$.
- Crear el video de demo (ver sección 8.4).
- Preparar la presentación final (20 min + 10 min preguntas).

7.3 Entregables

Entregable 1: Repositorio con Dos Ramas

Rama main: El monolito original. No se modifica.

Rama hexagonal: El sistema completamente refactorizado. Debe incluir:

- [COMPARATIVA.md](#): Tabla comparativa antes/después mostrando: líneas de código por módulo, tiempo de ejecución de pruebas (antes: N/A o lento, después: milisegundos), número de dependencias por capa.
- [ARQUITECTURA.md](#): Diagrama completo del sistema refactorizado.
- Historial de commits limpio que refleje las 5 fases (el docente revisará las fechas de commits).

Entregable 2: Informe Técnico Final

Documento PDF de mínimo 30 páginas con la siguiente estructura OBLIGATORIA:

- **Análisis del Monolito (Fase 1):** Diagrama de dependencias del monolito con anotaciones de cada violación encontrada.
- **Decisiones de Diseño:** Para cada entidad, objeto de valor y puerto, explicar POR QUÉ se diseñó así. No describir QUÉ hace el código, sino POR QUÉ se tomó esa decisión de diseño.
- **Catálogo de Puertos completo:** Tabla con cada puerto, su intención de negocio, métodos que expone, y todos los adaptadores que lo implementan.
- **Evidencia de evolución:** Documentar el proceso de agregar el segundo canal de entrada y el segundo adaptador de salida. Mostrar explícitamente que el código del dominio y de los casos de uso NO fue modificado durante este proceso.

- **Análisis cuantitativo:** Comparar: cobertura de pruebas antes vs. después, tiempo de ejecución del test suite, número de archivos que se deben modificar para agregar una nueva funcionalidad en el monolito vs. en la arquitectura hexagonal.
- **Reflexión crítica:** ¿En qué casos la Arquitectura Hexagonal NO sería la mejor opción? ¿Para qué tipo de proyectos podría ser sobre-ingeniería? Esta reflexión debe ser honesta y fundamentada.
- **Conclusiones y lecciones aprendidas.**

Entregable 3: Presentación a la Clase

Presentación de 20 minutos + 10 minutos de preguntas abiertas. Estructura:

- **Historia del problema (3 min):** Mostrar el monolito original con sus problemas. Ser concreto: mostrar el código y señalar las violaciones.
- **El proceso de refactorización (5 min):** Narrar cómo se abordó la refactorización. ¿Por dónde se empezó? ¿Qué fue lo más difícil? Mostrar un commit de cada fase.
- **Demo en vivo del sistema refactorizado (7 min):** Demostrar los 3 modos de ejecución (memoria, PostgreSQL, segundo DB). Demostrar el segundo canal de entrada.
- **Análisis cuantitativo (3 min):** Mostrar la tabla comparativa. Hablar de métricas: cobertura, velocidad de tests, facilidad de cambio.
- **Preguntas (2 min por pregunta):** El docente puede preguntar a cualquier integrante sobre cualquier línea del código. Si un integrante no puede responder preguntas básicas sobre el código, se aplicará penalización del 15% a la nota individual.

Entregable 4: Video de Demostración (Opcional genera 5 puntos adicionales a la nota final del corte)

Video de máximo 10 minutos (subido a YouTube o Drive, con link en el README) que demuestre:

- Los tests del dominio corriendo en menos de 2 segundos.
- Arranque del sistema con `MODO_REPOSITORIO=memoria` y ejecución de pruebas de integración.
- Arranque del sistema con `MODO_REPOSITORIO=postgres` (con `docker-compose up`) y verificación de persistencia.
- Arranque del sistema con el segundo repositorio (MongoDB u otro) y demostración de que los mismos tests pasan.
- Demostración del segundo canal de entrada funcionando.

7.4 Rúbrica de Evaluación

Criterio	Peso	Indicadores de Logro
Análisis del Monolito (Fase 1)	10%	Diagrama de dependencias correcto. Todas las violaciones identificadas con número de línea. Diseño propuesto coherente y bien justificado.
Calidad del Dominio (Fase 2)	20%	Cero imports prohibidos (script automático). Reglas de negocio en entidades, no en casos de uso. 20+ pruebas unitarias, todas pasan.
Correctitud de la Refactorización (Fases 3-4)	20%	Los 6+ casos de uso funcionan. Inyección de dependencias correcta. Los casos de uso no dependen de adaptadores concretos.
Evidencia de Evolución	20%	Segundo canal de entrada funciona correctamente. Segundo adaptador de salida implementado. CERO líneas del dominio y casos de uso fueron modificadas para agregar estos.
Suite de Pruebas	Opcional	Cobertura $\geq 85\%$. Video demuestra los 3 modos de ejecución.
Informe y Presentación	30%	Decisiones de diseño justificadas (no solo descritas). Análisis cuantitativo presente. Reflexión crítica honesta. Demo en vivo funciona.

8. Banco de Preguntas de Reflexión y Sustentación

Durante la sustentación oral, cualquiera de las siguientes preguntas puede ser formulada por el docente a cualquier integrante del grupo. Se espera una respuesta fundamentada con referencia al código:

8.1 Preguntas Conceptuales

13. ¿Cuál es la diferencia entre un Puerto de Entrada y un Adaptador de Entrada? ¿Podrían fusionarse en una sola clase? ¿Por qué no?
14. Si se decide migrar de PostgreSQL a MongoDB, ¿qué archivos exactamente del proyecto deben modificarse? ¿Cuántos son? Señale cada uno en el repositorio.
15. ¿En qué lado del hexágono se ubicaría un sensor IoT que envía datos de temperatura? Justifique su respuesta.
16. ¿Puede el Dominio lanzar excepciones? ¿Las excepciones de dominio pueden mencionar tecnologías como HTTP 400 o SQL Constraint Violation? Justifique.
17. ¿Cuál es la diferencia entre un Fake y un Mock en el contexto de las pruebas de Arquitectura Hexagonal? ¿Cuándo se usa cada uno?

8.2 Preguntas de Diseño y Código

18. Muéstreme el flujo completo de una petición HTTP POST para crear un registro desde el controlador FastAPI hasta el guardado en base de datos. Explique qué hace cada clase.
19. ¿Por qué los Puertos de Salida son interfaces en el módulo de puertos/ y no en el módulo de adaptadores/? ¿Qué pasaría si estuvieran en adaptadores/?
20. Si mañana se requiere agregar autenticación JWT, ¿qué capa(s) se modificarían? ¿El dominio necesita saber que existe JWT? Justifique.
21. ¿Qué es un DTO (Data Transfer Object) y por qué es importante en la frontera entre el Adaptador de Entrada y el Puerto de Entrada?
22. Muéstreme cómo se configura la inyección de dependencias en el contenedor. ¿Qué sucedería si se inyectara directamente la implementación concreta en el constructor del Caso de Uso?

8.3 Preguntas de Evolución y Escalabilidad

23. ¿Cómo evolucionaría este sistema si se necesita soportar multi-tenancy (múltiples organizaciones con datos aislados)?
24. ¿Cómo se convertiría este sistema en un microservicio independiente? ¿Qué cambiaría en la arquitectura? ¿Qué permanecería igual?
25. ¿En qué escenarios la Arquitectura Hexagonal podría ser innecesariamente compleja? ¿Para qué tamaño de proyecto no recomendaría usarla?

26. Si el sistema necesita soportar procesamiento en tiempo real de miles de transacciones por segundo, ¿cómo adaptaría la arquitectura hexagonal para soportar esto sin violar sus principios?

9. Errores Comunes y Guía de Diagnóstico

La siguiente tabla documenta los errores más frecuentes observados en implementaciones de Arquitectura Hexagonal por estudiantes. Use esta guía como checklist antes de entregar su trabajo:

Error	Síntoma Observable	Corrección
Lógica de negocio en el controlador	El controlador FastAPI contiene if/else para validar reglas. Ej: 'if monto > limite_diario'	Mover la validación a la Entidad del Dominio como método. El controlador solo traduce HTTP → DTO → Puerto.
Puertos nombrados por tecnología	Los puertos se llaman PostgresRepository, KafkaPort, EmailService	Renombrar por intención: TransaccionRepositorioPort, NotificadorPort. La tecnología solo aparece en el adaptador.
El Caso de Uso implementa lógica de dominio	El caso de uso calcula descuentos, verifica límites o aplica fórmulas de negocio directamente	Las reglas viven en las Entidades. El Caso de Uso solo orquesta: recupera entidad → invoca método → persiste.
El Dominio importa SQLAlchemy	'from sqlalchemy import Column' en dominio/entidades.py	El modelo ORM debe estar SOLO en adaptadores/salida/. El dominio usa exclusivamente Pydantic y stdlib.
Contenedor de DI instancia adaptadores en el caso de uso	El Caso de Uso hace 'self.repo = RepositorioPostgres()' en su constructor	Los adaptadores se inyectan desde afuera (contenedor). El constructor del Caso de Uso solo recibe la interfaz abstracta.
Ausencia de adaptador en memoria	Para correr cualquier prueba se necesita PostgreSQL corriendo	Implementar RepositorioMemoria que permita pruebas en milisegundos sin infraestructura.
Puerto de salida dentro del paquete de adaptadores	La interfaz ABC de persistencia está en adaptadores/salida/__init__.py	Los puertos pertenecen a puertos/salida.py. Los adaptadores IMPLEMENTAN los puertos, no los definen.

10. Recursos Bibliográficos y Herramientas

10.1 Bibliografía Obligatoria

27. **Cockburn, A. (2005)**. Hexagonal Architecture. *alistair.cockburn.us*. El artículo original que define el patrón Ports & Adapters.
28. **Martin, R. C. (2017)**. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall. Capítulos 17-22: Screaming Architecture, Clean Architecture, Presenters and Humble Objects.
29. **Evans, E. (2003)**. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley. Capítulos 1-4: Pone las bases del modelado de dominio.
30. **Vernon, V. (2013)**. *Implementing Domain-Driven Design*. Addison-Wesley. Capítulos 3-4: Architecture y Contextual Integration.

10.2 Bibliografía Complementaria

- **Fowler, M. (2002)**. *Patterns of Enterprise Application Architecture*. Addison-Wesley. Patrones Repository, Service Layer y Data Mapper.
- **Freeman, S. & Pryce, N. (2009)**. *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley. Enfoque TDD que complementa el diseño hexagonal.
- **Richardson, C. (2018)**. *Microservices Patterns*. Manning Publications. Capítulos sobre evolución de monolito a microservicios.

10.3 Herramientas Recomendadas

Herramienta	Uso en el Proyecto
pytest + pytest-cov	Suite de pruebas y medición de cobertura de código
pydantic v2	Modelado del dominio con validación de tipos en Python
FastAPI	Framework para adaptadores de entrada REST
SQLAlchemy 2.0	ORM para adaptadores de salida PostgreSQL
Docker + docker-compose	Infraestructura efímera para pruebas end-to-end
PlantUML o draw.io	Diagramación UML y arquitectónica
testcontainers-python	Contenedores Docker manejados desde Python para tests
dependency-injector	Librería Python para contenedores de inyección de dependencias

black + ruff	Formateo y linting de código Python
pre-commit	Hooks de Git para verificar calidad antes de cada commit

11. Nota Final del Docente

Mensaje para los Estudiantes

La Arquitectura Hexagonal no es un fin en sí mismo. Es una herramienta que, cuando se aplica con criterio, produce sistemas que sobreviven al tiempo y a los cambios de requisitos. En el mercado laboral, la capacidad de razonar sobre fronteras arquitectónicas y de desacoplar responsabilidades es una de las habilidades más valoradas y menos comunes.

Durante el desarrollo de estos trabajos, se enfrentarán a momentos donde la tentación de 'tomar el atajo' —poner la lógica directamente en el controlador, o hacer que el dominio importe SQLAlchemy— será muy grande. Resístanla. Ese es exactamente el momento donde se aprende.

El criterio de evaluación más importante no es que el código funcione, sino que funcione de la manera correcta. Un sistema que funciona pero con una arquitectura acoplada es simplemente deuda técnica aplazada.

Se les desea mucho éxito en el proceso. Las preguntas son bienvenidas durante todo el desarrollo del trabajo.

Última actualización: 2026

Basado en: Cockburn (2005), Martin (2017), Evans (2003)